Guía de Teoría: Actualización de Bases de Datos (RA UPDATE)

Departamento de Informática

March 13, 2026

Introducción

Esta guía detalla las técnicas y comandos esenciales para la actualización, manipulación y saneamiento de datos en sistemas gestores de bases de datos. Se abordarán desde los escenarios básicos de inicialización hasta técnicas avanzadas de migración, gestión de colisiones, saneamiento profundo, auditoría y reestructuración del esquema.

1 PASO 0: El Escenario del Crimen (Script de Inicialización)

Para seguir esta guía, es necesario crear una base de datos de pruebas que contenga datos "sucios" y errores comunes de importación.

Instrucciones: Ejecuta el script adjunto `00_fugas_setup.sql` en tu servidor local para preparar el entorno de trabajo. Este script genera tablas de producción, tablas de *staging* y registros con inconsistencias deliberadas.

Listing 1: Script de inicialización: `00_fugas_setup.sql`

```sql
DROP DATABASE IF EXISTS FugaTech_Forensic;
CREATE DATABASE FugaTech_Forensic CHARACTER SET utf8mb4;
USE FugaTech_Forensic;

-- 1. Tabla de Producción (Clientes)
CREATE TABLE clientes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100),
    email VARCHAR(100),
    telefono VARCHAR(50),
    credito DECIMAL(10,2) DEFAULT 0.00,
    activo BOOLEAN DEFAULT 1
);

-- 2. Tabla de Ventas (Para integridad referencial)
CREATE TABLE ventas (
    id INT AUTO_INCREMENT PRIMARY KEY,
    id_cliente INT,
    fecha DATETIME DEFAULT NOW(),
    total DECIMAL(10,2),
```

```sql
    FOREIGN KEY (id_cliente) REFERENCES clientes(id) ON DELETE RESTRICT
);

-- 3. Cuentas Bancarias (Para ejemplos de subconsultas)
CREATE TABLE cuentas_bancarias (
    id INT AUTO_INCREMENT PRIMARY KEY,
    id_cliente INT,
    iban VARCHAR(34),
    fraude BOOLEAN DEFAULT 0,
    FOREIGN KEY (id_cliente) REFERENCES clientes(id)
);

-- 4. Catalogo e Importación (Para ejemplos de JOIN en UPDATE)
CREATE TABLE catalogo (
    referencia VARCHAR(50) PRIMARY KEY,
    nombre VARCHAR(100),
    precio DECIMAL(10,2),
    fecha_actualizacion DATETIME
);

CREATE TABLE importacion_tarifas (
    referencia VARCHAR(50) PRIMARY KEY,
    nuevo_precio DECIMAL(10,2)
);

-- 5. Tabla "Sucia" (Staging - Importación de Excel/CSV chapucero)
CREATE TABLE import_raw (
    raw_id INT,
    raw_data_str VARCHAR(255), -- Datos mezclados: "Nombre, Email"
    raw_phone VARCHAR(50)
);

-- 6. Logs (Para borrado masivo)
CREATE TABLE sys_logs (
    id INT AUTO_INCREMENT PRIMARY KEY,
    msg TEXT,
    created_at DATETIME
);

-- --- INSERCIÓN DE DATOS "ROTOS" Y SUCIOS ---

INSERT INTO clientes (nombre, email, telefono, credito) VALUES
('Juan Gomez', 'juan.gomez@empresa.local', '+34 600-111-222', 100.00),
('Maria Lopez', 'maria.lopez@gmail,com', '600 222 333', 50.50),
('Carlos Ruiz', 'carlos.ruiz@hotmail.com', '0034600333444', 0.00),
('Admin Sys', 'admin@fugatech.com', NULL, 9999.99);

INSERT INTO ventas (id_cliente, total) VALUES (1, 500.00), (2, 120.00);

INSERT INTO cuentas_bancarias (id_cliente, iban, fraude) VALUES (3, 'ES12345678',
    1);

INSERT INTO catalogo VALUES ('REF-001', 'Switch Cisco', 200.00, '2020-01-01');
```

```sql
INSERT INTO importacion_tarifas VALUES ('REF-001', 250.00);

INSERT INTO import_raw VALUES
(101, ' PEDRO MARTINEZ , pedro@mail.com', '+34 666-666-666'),
(102, 'Ana sanz, ana@test.com', '666777888');

INSERT INTO sys_logs (msg, created_at) VALUES
('Error 404', '2020-01-01 10:00:00'),
('Login OK', '2021-05-05 12:00:00'),
('Warning', NOW());
```

2 El Entorno de Trabajo y Seguridad

Para manipular grandes volúmenes de datos, es crucial controlar tanto la terminal como los clientes gráficos, conociendo sus mecanismos de seguridad.

2.1 Ejecución desde la Terminal y Modo Seguro

Tip de terminal vs GUI

El saneamiento masivo no debe hacerse "a mano". El **Modo Seguro** es una restricción del **cliente** (como MySQL Workbench), no del servidor. Por tanto, los scripts ejecutados directamente por terminal (`mysql < script.sql`) suelen ejecutarse sin estas restricciones, evitando bloqueos por falta de memoria RAM comunes en entornos gráficos al renderizar scripts de gran tamaño.

2.2 MySQL Workbench y la directiva *Safe Updates*

Workbench lanza el **Error Code: 1175** si intentas un `UPDATE` sin una PK en el `WHERE`. Como "forenses", debemos sortearlo:

Listing 2: Gestión de Safe Updates: `01_safe_updates.sql`

```sql
-- Intenta esto primero para ver el Error 1175
UPDATE clientes SET email = REPLACE(email, ',', '.') WHERE email LIKE '%@%,%';

-- Solución forense:
SET SQL_SAFE_UPDATES = 0;

UPDATE clientes
SET email = REPLACE(email, ',', '.')
WHERE email LIKE '%@%,%';

SET SQL_SAFE_UPDATES = 1;
```

Reflexión 0

¿Qué sentido tiene el modo seguro y por qué viene activado por defecto en los entornos de aprendizaje y clientes gráficos?

3 Integridad Referencial y Restricciones

Las tablas no son islas; están unidas por claves foráneas (FK) que evalúan reglas cada vez que ejecutamos DML.

Recomendación

Para una comprensión profunda de las FKs, consulta la guía específica de **Integridad Referencial** disponible en la carpeta `/home/vicdejuan/BasesDeDatos/Teoria/UT4/01_On_delete_or_update/`.

3.1 Comportamientos ante UPDATE y DELETE

- **RESTRICT / NO ACTION:** El escudo principal. Bloquea la sentencia si existen registros dependientes.
- **CASCADE:** El bisturí automático.
  - `ON DELETE CASCADE`: Borrar un padre fulmina automáticamente a sus hijos (Peligro de pérdida masiva).
  - `ON UPDATE CASCADE`: Si el ID principal cambia (ej. corregir un DNI mal metido), el cambio se propaga a todas las tablas dependientes.
- **SET NULL:** Conservación de históricos. Si eliminamos un comercial, no borramos sus ventas (`CASCADE`), sino que las dejamos "huérfanas" pero intactas colocando `NULL` en su FK.

3.2 Desactivación de Claves Foráneas para Ingesta

En migraciones, importamos tablas en órdenes aleatorios. La técnica estándar es suspender temporalmente la comprobación:

Listing 3: Control de FKs: `02_fk_checks.sql`

```sql
-- 1. Suspendemos la validación de claves foráneas
SET FOREIGN_KEY_CHECKS = 0;

-- 2. Realizamos vaciados (TRUNCATE) o inserciones masivas sin importar el orden
TRUNCATE TABLE ventas;
TRUNCATE TABLE clientes;

-- 3. Volvemos a activar la validación (¡Obligatorio!)
SET FOREIGN_KEY_CHECKS = 1;
```

Reflexión 1

¿Cuál es el problema de eliminar el contenido de las tablas desactivando las claves foráneas (*foreign keys*)?

ADVERTENCIA

¡Ojo! Acabas de eliminar todos los datos de la base de datos. Antes de seguir con el resto de ejemplos, vuelve a cargar la parte del script de inserción.

4 Inserción Avanzada y Consolidación

La inserción fila a fila carece de utilidad en entornos de Big Data. Debemos dominar las inserciones masivas.

4.1 Motivación: ¿Cuándo necesitamos migrar datos?

- **Legacy Systems:** Pasar datos de una base antigua a una nueva estructura.
- **Consolidación:** Unificar bases tras una fusión de empresas.
- **ETL de staging:** Procesar datos de Excels externos antes de llevarlos a producción.

4.2 Migración con SELECT y Gestión de Colisiones

La técnica `INSERT INTO ... SELECT` permite transformar datos al vuelo.

Listing 4: Migración con SELECT: `03_insert_select.sql`

```sql
-- 1. Creamos la tabla de destino (Estructura optimizada para marketing)
-- Nota: No tiene por qué ser idéntica a la tabla origen.
CREATE TABLE IF NOT EXISTS clientes_vip (
    cliente_id INT PRIMARY KEY,
    nombre_completo VARCHAR(150),
    email_contacto VARCHAR(100),
    potencial_compra DECIMAL(10,2),
    fecha_inclusion DATETIME DEFAULT NOW()
);

-- 2. Limpieza previa (para que el ejemplo sea repetible)
TRUNCATE TABLE clientes_vip;

-- 3. INSERT INTO ... SELECT
-- Extraemos datos de 'clientes', los transformamos y los insertamos en '
    clientes_vip'
INSERT INTO clientes_vip (cliente_id, nombre_completo, email_contacto,
    potencial_compra)
SELECT
    id,
    TRIM(nombre), -- Saneamiento: quitamos espacios
    LOWER(REPLACE(email, ',', '.')), -- Saneamiento: arreglamos comas en emails
    credito * 1.20 -- Transformación: calculamos potencial (un 20% más)
FROM clientes
WHERE activo = 1
  AND credito > 50; -- Solo clientes con crédito significativo

-- 5. Verificación
SELECT * FROM clientes_vip;
```

Punto Crítico: Restricciones UNIQUE

El motor evalúa las restricciones de `UNIQUE` para cada fila. Si hay una colisión, la sentencia fallará atómicamente a menos que usemos `IGNORE` o `ON DUPLICATE KEY UPDATE` (*Upsert*).

Listing 5: Estrategia Upsert: `04_upsert.sql`

```
-- Tecnica de UPSERT aplicada a la tabla de clientes.
-- Si el cliente con ID 1 no existe, se crea con los datos proporcionados.
-- Si el ID 1 ya existe, se actualiza su credito (anadiendo 100) y se marca como
    activo.

INSERT INTO clientes (id, nombre, email, telefono, credito, activo)
VALUES (1, 'Juan Gomez', 'juan.gomez@empresa.local', '+34 600-111-222', 100.00, 1)
ON DUPLICATE KEY UPDATE
    credito = credito + 100.00,
    activo = 1;
```

Reflexión 2

¿Qué objetivo técnico perseguimos al combinar `TRIM()`, `UPPER()` y `CONCAT()` durante una importación masiva?

Reflexión 3

¿Qué ventajas aporta realizar una consulta (`SELECT`) dentro de una sentencia de inserción (`INSERT`) en lugar de hacerlo en dos pasos separados?

5 UPDATE: Saneamiento Profundo

5.1 Funciones de cadena y Normalización

Los datos heredados suelen tener espacios excesivos o formatos incorrectos. Usamos funciones nativas:

Listing 6: Normalización de datos: `05_phone_norm.sql`

```
-- Supongamos telefonos importados como "+34 600 123 456" o "0034 600-123-456"
UPDATE clientes
SET telefono = REPLACE(REPLACE(REPLACE(telefono, '+34', ''), '0034', ''), '-', '')
WHERE telefono LIKE '+34%' OR telefono LIKE '0034%' OR telefono LIKE '%-%';

-- Segundo pase: limpiar posibles espacios generados
UPDATE clientes SET telefono = TRIM(telefono);
```

5.2 UPDATE con subconsultas y JOINs

Para corregir datos basándonos en otras tablas, podemos usar subconsultas o la sintaxis de JOIN múltiple de MySQL.

Listing 7: Update con Subconsultas: `06_update_subquery.sql`

```sql
-- Bloquear a los clientes cuyas cuentas bancarias asociadas esten marcadas como
    fraudulentas
UPDATE clientes
SET activo = 0
WHERE id IN (
    SELECT id_cliente FROM cuentas_bancarias WHERE fraude = 1
);
```

Listing 8: Update con JOIN: `07_update_join.sql`

```sql
-- Actualizar los precios del catálogo cruzándolos con una tabla temporal de
    nuevas tarifas
UPDATE catalogo c
INNER JOIN importacion_tarifas t ON c.referencia = t.referencia
SET c.precio = t.nuevo_precio, c.fecha_actualizacion = NOW();
```

5.3 Lógica Condicional Compleja (CASE)

Listing 9: Lógica condicional: `08_case_logic.sql`

```sql
-- Lógica condicional dentro del UPDATE
-- Ejemplo: Normalizar prefijos internacionales basándose en patrones
UPDATE clientes
SET telefono = CASE
    WHEN email LIKE '%.local' THEN CONCAT('+34 ', telefono)
    WHEN nombre LIKE 'Admin%' THEN '+00 000000000'
    ELSE telefono
END
WHERE activo = 1;
```

Reflexión 4

¿Por qué es preferible usar un `CASE` dentro de un único `UPDATE` en lugar de ejecutar varios `UPDATE` con diferentes cláusulas `WHERE`?

5.4 El Problema de los NULLs

Concatenar con `NULL` devuelve `NULL`. Usamos `IFNULL()` o `COALESCE()` para evitarlo.

Listing 10: Gestión de NULLs: `09_null_handling.sql`

```sql
-- ==============================================================================
-- GESTIÓN DE NULLS: IFNULL vs COALESCE
-- ==============================================================================

-- 0. Preparación de datos de prueba (Escenarios de nulos)
INSERT INTO clientes (nombre, email, telefono, credito) VALUES
('Sujeto A', NULL, '600-000-001', 10.00), -- Solo teléfono
('Sujeto B', 'b@test.com', NULL, 20.00), -- Solo email
('Sujeto C', NULL, NULL, 0.00), -- Sin datos de contacto
('Sujeto D', 'd@test.com', '600-000-004', 5.00); -- Datos completos
```

```sql
-- 1. IFNULL: El "plan B" binario (solo 2 argumentos)
-- Si el teléfono es NULL, pone 'DESCONOCIDO'.
UPDATE clientes
SET telefono = IFNULL(telefono, 'DESCONOCIDO')
WHERE nombre LIKE 'Sujeto%';

-- 2. COALESCE: El "Selector de Prioridades" (N argumentos)
-- Devuelve el primer valor NO NULO de una lista.
-- Escenario: 1. Email (Preferido), 2. Teléfono (Backup), 3. Texto por defecto.
SELECT
    nombre,
    COALESCE(email, telefono, 'ILOCALIZABLE') AS contacto_urgente
FROM clientes
WHERE nombre LIKE 'Sujeto%';

-- 3. Evitar que una concatenación "rompa" el resultado
-- Problema: 'Sr. ' + NULL = NULL (El NULL es como un agujero negro)
SELECT
    CONCAT('Expediente: ', COALESCE(nombre, 'ANÓNIMO')) AS info_sujeto
FROM clientes;
```

Reflexión 5

Si intentamos concatenar el nombre 'Juan' con un teléfono que es `NULL`, ¿qué resultado obtenemos y cómo lo evitamos?

5.5 Saneamiento de Fechas y Tipos (CAST)

Listing 11: Saneamiento de tipos: `10_date_saneamiento.sql`

```sql
-- Conversión de tipos y saneamiento de fechas
-- Ejemplo: Convertir texto '01/01/2020' a tipo DATETIME
UPDATE sys_logs
SET created_at = STR_TO_DATE('12/03/2026 10:30:00', '%d/%m/%Y %H:%i:%s')
WHERE id = 1;

-- Uso de CAST para asegurar tipos numéricos en importaciones de cadenas
UPDATE clientes
SET credito = CAST('150.50' AS DECIMAL(10,2))
WHERE nombre = 'Carlos Ruiz';
```

6 Reestructuración Quirúrgica: Modificación del Esquema (DDL)

6.1 Añadir y Eliminar Columnas

Listing 12: Modificación de columnas: `11_modify_columns.sql`

```sql
-- Añadir una columna de NIF después del nombre
```

```sql
ALTER TABLE clientes ADD COLUMN nif VARCHAR(12) AFTER nombre;

-- Eliminar una columna que ya no es necesaria (ejemplo conceptual)
-- ALTER TABLE sys_logs DROP COLUMN metadata_obsoleta;
```

6.2 Modificar Tipos de Datos y Nombres

Listing 13: Ajuste de tipos de datos: `12_modify_types.sql`

```sql
-- Ampliar la longitud permitida para el nombre del cliente
ALTER TABLE clientes MODIFY COLUMN nombre VARCHAR(250);

-- Ejemplo de cambio de nombre y tipo (Sintaxis MySQL)
-- ALTER TABLE clientes CHANGE COLUMN nombre nombre_completo VARCHAR(200);
```

6.3 Imponer la Integridad: Unicidad y Obligatoriedad

Listing 14: Añadir Unique y Not Null: `13_add_constraints.sql`

```sql
-- Forzar la unicidad de los correos electrónicos para evitar duplicados en el
    futuro
ALTER TABLE clientes ADD CONSTRAINT uq_email UNIQUE (email);

-- Convertir una columna en obligatoria (NOT NULL)
-- ¡Atención! Primero saneamos los nulos existentes para evitar errores de
    restricción
UPDATE clientes SET nif = '00000000T' WHERE nif IS NULL;

ALTER TABLE clientes MODIFY nif VARCHAR(12) NOT NULL;
```

Reflexión 8

¿Por qué suele fallar un `ALTER TABLE ... MODIFY ... NOT NULL` en una base de datos que ya contiene datos? ¿Qué paso previo es obligatorio realizar?

6.4 Validación de Reglas de Negocio (CHECK)

Listing 15: Restricciones de validación: `14_add_checks.sql`

```sql
-- Impedir que el crédito del cliente sea negativo mediante una restricción de
    validación
ALTER TABLE clientes ADD CONSTRAINT chk_credito_positivo CHECK (credito >= 0);

-- Validación compleja: Asegurar que el NIF tenga 8 números y una letra (formato b
    ásico)
ALTER TABLE clientes ADD CONSTRAINT chk_nif_formato CHECK (nif REGEXP '^[0-9]{8}[A
    -Z]$');
```

7 Optimización y Rendimiento

7.1 SARGability: El peligro de las funciones en el WHERE

Usar funciones sobre columnas indexadas impide al motor realizar un *Index Seek*.

Listing 16: Rendimiento y SARGability: `15_performance_sarg.sql`

```sql
-- Rendimiento y SARGability (Search ARgument ABLE)
-- MAL: Evita funciones en la columna del WHERE (Invalida índices)
SELECT * FROM clientes WHERE UPPER(email) = 'JUAN@MAIL.COM';

-- BIEN: Compara contra la columna directamente (SARGable)
SELECT * FROM clientes WHERE email = 'juan@mail.com';

-- MAL: TRIM() en el WHERE obliga a un Full Table Scan al motor
UPDATE clientes SET activo = 0 WHERE TRIM(nombre) = 'Juan Gomez';
```

Reflexión 6

¿Qué diferencia de rendimiento existe entre `WHERE UPPER(email) = '...'` y `WHERE email = '...'` si la columna email está indexada?

8 Eliminación de Datos y Auditoría

8.1 DELETE vs TRUNCATE: El Duelo de Gigantes

- **DELETE:** Operación DML fila a fila. Comprueba restricciones, dispara triggers y escribe cada fila en el log de transacciones. **Mantiene** el `AUTO_INCREMENT`. Es muy lento en tablas masivas.
- **TRUNCATE:** Operación DDL estructural. Destruye y recrea la tabla. Instantánea, no satura el log, pero **resetea** el `AUTO_INCREMENT` y no activa triggers ni admite `WHERE`.

Listing 17: Borrado condicional: `16_delete_date.sql`

```sql
-- Purga de registros obsoletos basandose en fechas (Ej. Ley de Proteccion de
    Datos)
DELETE FROM sys_logs
WHERE created_at < DATE_SUB(NOW(), INTERVAL 5 YEAR);
```

Listing 18: Vaciado estructural: `17_truncate.sql`

```sql
-- Vaciado inmediato y reseteo del auto_incremento de la tabla de importacion tras
    procesarla
TRUNCATE TABLE import_raw;
```

8.2 Verificación Post-Intervención

Listing 19: Auditoría y Verificación: `18_audit_verify.sql`

```sql
-- Verificación post-intervención mediante agregación
-- Comprobar si han quedado correos repetidos tras el UPDATE
SELECT email, COUNT(*) as repetidos
FROM clientes
GROUP BY email
HAVING COUNT(*) > 1;

-- Verificar la distribución de datos tras un saneamiento masivo
SELECT activo, COUNT(*) as total
FROM clientes
GROUP BY activo;
```

Reflexión 7

¿Por qué es vital usar `GROUP BY` y `HAVING` tras un proceso de limpieza de duplicados?

9 Respuestas a las Reflexiones e Investigación

Respuesta 0: El Escudo de Seguridad

Evita borrados o modificaciones accidentales masivas por errores tipográficos en clientes gráficos.
Pista de investigación: Busca desastres reales como el de *GitLab* en 2017.

Respuesta 1: Integridad y Dependencias Circulares

Al desactivar FKs, el motor permite borrar registros que tienen dependencias activas, lo que puede dejar datos "huérfanos". Además, es la única forma de vaciar tablas con **dependencias circulares** (ej. un laboratorio dirigido por un investigador que a su vez pertenece a ese laboratorio).
Investiga: Grafos de Dependencia y herramientas como *Flyway*.

Respuesta 2: ETL y Calidad de Búsqueda

Limpieza: `TRIM()` evita errores de duplicados por espacios invisibles.
Normalización: `UPPER()` asegura consistencia en búsquedas posteriores, crítico si el origen de datos no está controlado.

Respuesta 3: Atomicidad y Eficiencia

Evita mover datos hacia la aplicación cliente y garantiza que la transformación ocurra de forma atómica y rápida dentro del motor.
Investiga: Operaciones Set-based.

Respuesta 4: Eficiencia de Proceso

Reduce el tiempo de bloqueo de filas (*Row Locks*) y escanea la tabla una sola vez.
Pista de investigación: Investiga los **Bloqueos de Fila**.

Respuesta 5: El vacío del NULL

Obtenemos `NULL`. Se evita usando `COALESCE(tel, '')`.
Pista de investigación: Busca la **Lógica Tri-valuada** de SQL.

Respuesta 6: SARGability

`UPPER(email)` obliga a un *Full Table Scan*, mientras que la comparación directa permite un *Index Seek*.
Pista de investigación: Busca el término **SARGable queries**.

Respuesta 7: Validación de Calidad

Permite confirmar que no han quedado registros duplicados. Es el control de calidad final.
Pista de investigación: Investiga técnicas de **Data Profiling**.

Respuesta 8: El cerrojo de la obligatoriedad

Falla si existen filas con valores `NULL`. El paso previo es sanear esos nulos mediante un `UPDATE` antes de aplicar la restricción.